
Warranty GENE — Shrish task list

Programme: EVEglyphDesign — Warranty GENE **Document ID:** EgD-WGB-002 · **Key ID:** EgD-KEY-2026-07 **For:** Shrish · **From:** Dany **Repository you will work in:** [EVEglyphDesign/hawkins-twin-platform](#), specifically the `warranty-gene/` subtree. **Companion:** [Warranty GENE Sidecar Blueprint \(EgD-WGB-001\)](#).

Execute the tasks in order. Each task has a command block and a gate. Do not proceed to the next task until the gate passes. If a gate fails and it is not one of the three questions in Task 12, keep trying — the answer is on the previous task.

Canon rule — VIN is the equipment record number

The **VIN is the base code for every equipment record** in this system and every downstream ring. It is the primary key, the filename, the directory anchor when a record has children, and the identifier that survives every ring boundary (Ring 2 → Ring 3 hashes it, but the hash is deterministic on the VIN — no other identifier is used).

Consequences you enforce in the tasks below:

- Filenames under `vins/` are exactly `{VIN}.json` — the VIN itself as the filename. No prefix, no suffix, no dashes of your own making.
- Work-order and booking records carry the VIN in a `vin` field and are joined to the equipment record by VIN, never by an internal DMS id.
- The equipment record schema treats `identity.vin` as required and immutable. A VIN correction is a delete-and-add, not an in-place edit.
- Indexes always emit VIN lists (or objects whose primary key is `vin`). Never a DMS record number, never a dealer-internal serial.
- When PACCAR / SIR / SmartLINQ records arrive with their own identifiers (SIR ID, unit number, stock number), those become `identity.aliases[]` — the VIN stays the base code.

This is the load-bearing decision behind the equipment-record data standard. Get it wrong once and the industry-wide repository will not merge.

Task 1 • Get access

```
gh auth login          # use your own GitHub account, SSO into EVEglyphDesign if
                        prompted
gh auth status         # confirms who you are authenticated as
gh repo view EVEglyphDesign/hawkins-twin-platform --json name,visibility,pushedAt
```

Gate: `gh repo view` returns JSON with `"visibility": "PUBLIC"` and a recent `pushedAt`. If it 404s, message Dany with the exact error — this is a Task 12 blocker, not a task-level failure.

Task 2 • Fork the repository into your account

```
gh repo fork EVEglyphDesign/hawkins-twin-platform --clone --remote
cd hawkins-twin-platform
git remote -v
```

Gate: `git remote -v` shows `origin` pointing at your fork (`/hawkins-twin-platform`) and `upstream` pointing at `EVEglyphDesign/hawkins-twin-platform`.

Task 3 • Confirm the Warranty GENE subtree is intact in your fork

```
ls warranty-gene/
ls warranty-gene/schema/ warranty-gene/samples/ warranty-gene/mappings/ warranty-gene/
wireframe/ warranty-gene/blueprint/
cat warranty-gene/schema/rich-target.schema.json | python -m json.tool > /dev/null &&
echo "schema parses"
cat warranty-gene/samples/rich-targets.sample.json | python -m json.tool > /dev/null &&
echo "samples parse"
```

Gate: every `ls` prints a non-empty listing and both parse checks print their success line. If any file is missing, you cloned wrong — delete and redo Task 2.

Confirm the schema treats VIN as the base code:

```
python -c "import json; s=json.load(open('warranty-gene/schema/rich-target.schema.json')); print('vin required:', 'vin' in s.get('required', []) or 'vin' in s.get('properties', {}).get('identity', {}).get('required', []))"
```

If this prints `vin required: False`, message Dany — Task 12 question #2. The schema must anchor on VIN before you add any records.

Task 4 · Create the dealer-records subtree

You are adding a **new** subtree, `warranty-gene/dealer-records/`, that holds one file per VIN. Nothing in the existing `warranty-gene/` files gets touched.

```
mkdir -p warranty-gene/dealer-records/peterbilt-atlantic/{R01-halifax,R02-moncton,R03-saint-john,R04-dartmouth,R05-truro,R06-sydney,R07-fredericton,R08-yarmouth}/vins
mkdir -p warranty-gene/dealer-records/peterbilt-atlantic/{R01-halifax,R02-moncton,R03-saint-john,R04-dartmouth,R05-truro,R06-sydney,R07-fredericton,R08-yarmouth}/work-orders
mkdir -p warranty-gene/dealer-records/indexes
mkdir -p warranty-gene/dealer-records/scripts
touch warranty-gene/dealer-records/peterbilt-atlantic/R01-halifax/.gitkeep # repeat
for each rooftop
find warranty-gene/dealer-records -type d
```

Gate: `find` prints the eight rooftop directories, `indexes/`, and `scripts/`. Commit and push:

```
git add warranty-gene/dealer-records/
git commit -m "warranty-gene: seed dealer-records subtree with eight Peterbilt Atlantic rooftops"
git push origin main
```

Task 5 · Copy the two committed sample records into the sample rooftop

The repo already ships two records at `warranty-gene/samples/`. Copy them into a sample rooftop so the indexer has something to walk that matches the eventual real layout.

```
mkdir -p warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins
python <<'PY'
import json, pathlib
src = json.loads(pathlib.Path("warranty-gene/samples/rich-targets.sample.json").read_text())
out = pathlib.Path("warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins")
recs = src if isinstance(src, list) else src.get("records", [src])
for r in recs:
    vin = r.get("vin") or r.get("identity", {}).get("vin")
    if not vin: continue
    (out / f"{vin}.json").write_text(json.dumps(r, indent=2))
print("wrote", len(list(out.glob('*.json'))), "records")
PY
ls warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins/
```

Gate: the `ls` prints at least two files named `{VIN}.json` (the VIN as filename). Commit:

```
git add warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/  
git commit -m "warranty-gene/dealer-records: seed R00-sample from warranty-gene/  
samples/"  
git push
```

Task 6 · Write the record validator

Create `warranty-gene/dealer-records/scripts/validate_records.py`. It walks every `.json` file under `warranty-gene/dealer-records/peterbilt-atlantic/ /vins/` and validates each against `warranty-gene/schema/rich-target.schema.json`. Red exit on any failure.

```
#!/usr/bin/env python3  
"""Validate every VIN record under dealer-records/ against the canon schema."""  
import json, sys, pathlib  
from jsonschema import Draft202012Validator  
  
ROOT = pathlib.Path(__file__).resolve().parents[3]  
SCHEMA = json.loads((ROOT / "warranty-gene/schema/rich-  
target.schema.json").read_text())  
VINS = list((ROOT / "warranty-gene/dealer-records/peterbilt-atlantic").glob("*/vins/  
*.json"))  
v = Draft202012Validator(SCHEMA)  
errors = 0  
for f in VINS:  
    rec = json.loads(f.read_text())  
    for e in v.iter_errors(rec):  
        errors += 1  
        print(f"FAIL {f.relative_to(ROOT)}: {e.message}")  
    if f.stem != (rec.get("vin") or rec.get("identity", {}).get("vin")):  
        errors += 1  
        print(f"FAIL {f.relative_to(ROOT)}: filename does not match record VIN")  
print(f"checked {len(VINS)} records, {errors} errors")  
sys.exit(1 if errors else 0)
```

Run it:

```
pip install jsonschema  
python warranty-gene/dealer-records/scripts/validate_records.py
```

Gate: the script prints `checked N records, 0 errors` and exits 0. If it errors, either the sample record is missing a required field (fix the sample in Task 5) or the schema is stricter than the sample (message Dany — this is Task 12 question #2).

Commit:

```
git add warranty-gene/dealer-records/scripts/validate_records.py
git commit -m "warranty-gene/dealer-records: add record validator"
git push
```

Task 7 · Write the index builder

Create `warranty-gene/dealer-records/scripts/build_indexes.py` . It walks every VIN record and emits five index files under `warranty-gene/dealer-records/indexes/` .

```
#!/usr/bin/env python3
"""Rebuild the five indexes from the primary records."""
import json, pathlib
from collections import defaultdict

ROOT = pathlib.Path(__file__).resolve().parents[3]
BASE = ROOT / "warranty-gene/dealer-records"
IDX = BASE / "indexes"
IDX.mkdir(exist_ok=True)

by_family = defaultdict(list)
by_coverage = defaultdict(list)
by_service = defaultdict(list)
open_recalls = defaultdict(list)
uplift = defaultdict(list)

for f in (BASE / "peterbilt-atlantic").glob("*/vins/*.json"):
    r = json.loads(f.read_text())
    vin = r.get("vin") or r.get("identity", {}).get("vin")
    fam = r.get("identity", {}).get("engine_family", "unknown")
    by_family[fam].append(vin)
    for c in r.get("coverage", []):
        by_coverage[f"{c.get('bucket','')}::{c.get('status','')}"].append(vin)
    ins = r.get("identity", {}).get("in_service_date", "")[:7]
    if ins: by_service[ins].append(vin)
    for rc in r.get("recalls_and_campaigns", []):
        if rc.get("status") == "open":
            open_recalls[rc.get("campaign","?")].append(vin)
    for u in r.get("uplift_candidates", []):
        uplift[u.get("tier","low")].append({
            "vin": vin, "wo_id": u.get("wo_id"),
            "line_item": u.get("line_item"), "bucket": u.get("bucket"),
        })

for name, data in [
    ("by-engine-family.json", by_family),
    ("by-coverage-status.json", by_coverage),
    ("by-in-service-date.json", by_service),
    ("open-recalls.json", open_recalls),
    ("uplift-candidates.json", uplift),
]:
    (IDX / name).write_text(json.dumps(data, indent=2, sort_keys=True))
    print(f"wrote {name} — {len(data)} keys")
```

Run it:

```
python warranty-gene/dealer-records/scripts/build_indexes.py
ls warranty-gene/dealer-records/indexes/
```

Gate: all five index files exist and each is valid JSON:

```
for f in warranty-gene/dealer-records/indexes/*.json; do python -m json.tool "$f" > /dev/null && echo "$f ok"; done
```

Every line prints `ok` . Commit:

```
git add warranty-gene/dealer-records/scripts/build_indexes.py warranty-gene/dealer-records/indexes/
git commit -m "warranty-gene/dealer-records: add index builder and initial indexes"
git push
```

Task 8 · Test the indexing round-trip

Prove that every VIN in the primary records appears in every index it should. Create `warranty-gene/dealer-records/scripts/test_indexing.py` :

```
#!/usr/bin/env python3
"""Round-trip test: every primary VIN appears in the indexes it should."""
import json, pathlib, sys

ROOT = pathlib.Path(__file__).resolve().parents[3]
BASE = ROOT / "warranty-gene/dealer-records"
IDX = BASE / "indexes"
idx = {p.name: json.loads(p.read_text()) for p in IDX.glob("*.json")}
errors = 0

def has(index_name, key, vin):
    entries = idx[index_name].get(key, [])
    return any((e == vin) or (isinstance(e, dict) and e.get("vin") == vin) for e in entries)

for f in (BASE / "peterbilt-atlantic").glob("*/vins/*.json"):
    r = json.loads(f.read_text())
    vin = r.get("vin") or r.get("identity", {}).get("vin")
    fam = r.get("identity", {}).get("engine_family", "unknown")
    if not has("by-engine-family.json", fam, vin):
        errors += 1; print(f"FAIL {vin} missing from by-engine-family[{fam}]")
    ins = r.get("identity", {}).get("in_service_date", "")[:7]
    if ins and not has("by-in-service-date.json", ins, vin):
        errors += 1; print(f"FAIL {vin} missing from by-in-service-date[{ins}]")

print(f"round-trip errors: {errors}")
sys.exit(1 if errors else 0)
```

```
python warranty-gene/dealer-records/scripts/test_indexing.py
```

Gate: prints `round-trip errors: 0` and exits 0.

Commit:

```
git add warranty-gene/dealer-records/scripts/test_indexing.py
git commit -m "warranty-gene/dealer-records: add round-trip indexing test"
git push
```

Task 9 • Wire the validate + test workflow

Create `.github/workflows/warranty-gene-dealer-records.yml` at the repo root of your fork:

```
name: warranty-gene dealer-records
on:
  push:
    paths:
      - "warranty-gene/dealer-records/**"
      - "warranty-gene/schema/**"
  pull_request:
    paths:
      - "warranty-gene/dealer-records/**"
      - "warranty-gene/schema/**"
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install jsonschema
      - run: python warranty-gene/dealer-records/scripts/validate_records.py
      - run: python warranty-gene/dealer-records/scripts/build_indexes.py
      - run: python warranty-gene/dealer-records/scripts/test_indexing.py
```

Commit and push:

```
git add .github/workflows/warranty-gene-dealer-records.yml
git commit -m "warranty-gene: CI validates records, builds indexes, tests round-trip"
git push
gh run watch
```

Gate: `gh run watch` finishes with a green check.

Task 10 • Measure repo size and project growth

Create `warranty-gene/dealer-records/scripts/measure_repo_size.py`:

```
#!/usr/bin/env python3
"""Measure current record footprint and project to 5k / 25k VINs."""
import pathlib, json

ROOT = pathlib.Path(__file__).resolve().parents[3]
FILES = list((ROOT / "warranty-gene/dealer-records/peterbilt-atlantic").glob("*/vins/*.json"))
sizes = [f.stat().st_size for f in FILES]
n = len(sizes); total = sum(sizes); avg = total / n if n else 0

print(f"# MEASURED-SIZE.md")
print(f"- VIN records: **{n}**")
print(f"- Total bytes: **{total:,}**")
print(f"- Average bytes per record: **{avg:,.0f}**")
print(f"- Largest record: **{max(sizes) if sizes else 0:,}** bytes")
print(f"- Projected at 5,000 VINs: **{5000*avg/1024/1024:,.1f} MB**")
print(f"- Projected at 25,000 VINs: **{25000*avg/1024/1024:,.1f} MB**")
```

```
python warranty-gene/dealer-records/scripts/measure_repo_size.py > warranty-gene/
dealer-records/MEASURED-SIZE.md
cat warranty-gene/dealer-records/MEASURED-SIZE.md
```

Gate: the file contains all six lines with real numbers, not zeros. Commit:

```
git add warranty-gene/dealer-records/scripts/measure_repo_size.py warranty-gene/dealer-
records/MEASURED-SIZE.md
git commit -m "warranty-gene/dealer-records: initial size measurement and projection"
git push
```

Task 11 · Prove upstream sync works — the whole point of the fork model

Canon updates on `EVEglyphDesign/hawkins-twin-platform` must flow into your fork by a plain merge. Test it now on an empty upstream:

```
git fetch upstream
git merge upstream/main --no-edit
git push
```

Gate: merge succeeds with `Already up to date.` or a fast-forward. If it produces a conflict on any file you did not create, message Dany — this is Task 12 question #3.

Record the upstream URL in the dealer-records README so it is durable. Write this file at `warranty-gene/dealer-records/README.md`:

```
# Warranty GENE – dealer records subtree
```

Forked from EVEglyphDesign/hawkins-twin-platform.

Pull canon updates with `git fetch upstream && git merge upstream/main`.

Every VIN is one file, filename is the VIN, one commit per change.

See EgD-WGB-002 – Shrish task list on GitHub Pages under
warranty-gene/shrish-setup/EgD-WGB-002-Shrish-Setup.pdf.

Then commit:

```
git add warranty-gene/dealer-records/README.md
git commit -m "warranty-gene/dealer-records: document upstream sync"
git push
```

Task 12 · Report back with one line and one URL

Message Dany with the URL of `warranty-gene/dealer-records/MEASURED-SIZE.md` on your fork and the workflow-run URL from Task 9. Nothing else.

If you got stuck, message Dany with one of these three, verbatim:

1. Task N gate failed. Command: {command}. Output: {exact error}.
2. Task 6 or 8 schema mismatch. Field: {name}. Sample record: {path}. Schema requires: {shape}.
3. Task 1, 2, or 11 access blocker. Error: {exact error}. Next step if resolved: {what you would run}.

If your question is not one of these three, keep working. Everything you need is already in the repo.

Provenance

- [EVEglyphDesign/hawkins-twin-platform](#) — the working repository
- [Warranty GENE Sidecar Blueprint \(EgD-WGB-001\)](#) — architecture
- [Warranty GENE schema v1](#) — record shape
- [Warranty GENE coverage tables](#)
- [Warranty GENE field mappings](#)
- [GitHub docs](#) — forking a repository
- [GitHub docs](#) — syncing a fork

© 2026 EVEglyphDesign. All rights reserved. Controlled copy. *Pour le bien-être du peuple.*